



Tyk Onboardin g

Sr. Customer Solutions Architect



Module 1

GraphQL

GraphQL

Definition (GraphQL Foundation):

- GraphQL is a query language for APIs and a runtime for fulfilling those queries with your existing data. It describes your data, lets clients request exactly what they need, and evolves safely over time.
- Key Benefits:
- Reduced network traffic – no over-fetching or under-fetching
- Flexible & language-agnostic – works with any backend
- Simplified data fetching – one request, structured response
- Easy to maintain – schema-based evolution
- Strong typing – catch errors early
- Great dev experience – ideal for mobile apps, public APIs, and microservices

GraphQL

The GraphQL Advantage:

Traditional REST API Call

To load user info + their posts:

1. GET /user/123
2. GET /user/123/posts

- Multiple round-trips
- Over-fetching / under-fetching

GraphQL

```
query {  
  user(id: "123") {  
    name  
    posts {  
      title  
      publishedAt  
    }  
  }  
}
```

- One request
- Only the data needed
- Structured JSON response

GraphQL

GraphQL uses a type system to define your API.

```
type User {  
  id: ID!  
  name: String!  
  email: String!  
  posts: [Post!]!  
}
```

```
type Post {  
  title: String!  
  publishedAt: String!  
}
```

- `String!` means it's required
- Relationships (e.g. `posts`) are embedded
- Tools can auto-generate documentation
- Clients get predictable responses
- Catches errors at development time

GraphQL

GraphQL is schema-based, so you can:

- Add new fields:

```
type User {  
  age: Int    # Added later  
}
```

- Deprecate old fields:

```
email: String @deprecated(reason: "Use contactEmail")
```

- No breaking changes
- Versionless API evolution
- Better long-term maintainability

GraphQL with Tyk

Native GraphQL Support

- No middleware required
- Fully compliant with latest GraphQL specs:
 - Queries – Fetching data
 - Mutations – Modifying data
 - Subscriptions – Real-time updates

What You Can Do with Tyk GraphQL:

- Securely expose existing GraphQL APIs
- Use Tyk's integrated engine to create a Universal Data Graph – combine multiple services into one unified GraphQL endpoint



Hands-On

Onboarding a GraphQL API

Onboarding a GraphQL API

Like a REST API, you can onboard via the Dashboard UI, Dashboard API or Gateway API for OSS

Steps:

1. Go to System Management → APIs
2. Click Add New API
3. Fill out Base Configuration:
 - Name, Type = GraphQL
 - Target URL (e.g. <https://countries.trevorblades.com/>)
4. Click Configure API
5. Choose Authentication Mode (e.g. Auth Token)
6. Click **SAVE** – Your GraphQL API is now active and secured.
7. Go to 'Playgrounds'
8. Send the following query:

```
query {  
  
  countries {  
  
    code  
  
    name  
  
    emoji  
  
  }  
  
}
```

Onboarding a GraphQL API via Operator

Onboarding a GraphQL API via Operator is similar to a Classic API Definition:

1. Run `git clone https://github.com/example-org/graphql-operator-tutorial`
2. Run `cd graphql-operator-tutorial`
3. Run `kubectl apply -f graphql-example.yaml -n tyk-cp`
4. Verify with `kubectl get tykapis -n tyk-cp`
5. Go to 'Playgrounds' in the Tyk Dashboard
6. Send the following query:

```
query {  
  
  countries {  
  
    code  
  
    name  
  
    emoji  
  
  }  
  
}
```

```

apiVersion: tyk.io/v1alpha1
kind: ApiDefinition
metadata:
  name: trevorblades
spec:
  name: trevorblades
  use_keyless: true
  protocol: http
  active: true
  proxy:
    target_url: https://countries.trevorblades.com
    listen_path: /trevorblades
    strip_listen_path: true
  graphql:
    enabled: true
    version: "2"
    execution_mode: proxyOnly
  schema: |
    directive @cacheControl(maxAge: Int, scope:

    enum CacheControlScope {
      PUBLIC
      PRIVATE
    }

    type Continent {
      code: ID!
      name: String!
      countries: [Country!]!
    }

    input ContinentFilterInput {
      code: StringQueryOperatorInput

```

- apiVersion
- kind
- metadata
 - name
- spec
 - name
 - use_keyless
 - protocol
 - active
 - proxy
 - target_url
 - listen_path
 - strip_listen_path
- graphql
 - enabled
 - version
 - execution_mode
 - schema
 - playground
 - enabled
 - path

Securing a GraphQL API via Operator

1. Use the example auth-sections.txt
2. Add section into graphql-example.yaml
3. Bonus: Run the command to apply the manifest

```
query {  
  countries {  
    code  
    name  
    emoji  
  }  
}
```

GraphQL Proxy Only

Definition:

GraphQL Proxy Only is a type of GraphQL API in Tyk that:

- Uses a **single data source**
- Has a **read-only schema**
- Automatically **loads schema** from the upstream using introspection queries

Key Characteristics:

- Upstream must support **introspection**
- Useful for exposing existing GraphQL services securely
- Supports all Tyk features like **policies** and **rate limiting**
- Minimal configuration — no schema stitching or data merging

Managing Schema

Keeping Your Schema in Sync:

Schema updates may be needed when your upstream GraphQL service changes.

The Tyk Dashboard displays the last sync timestamp for your API schema.

Core Settings Schema Playground Advanced Options



This schema has been automatically downloaded from the upstream provided and is read only. Schema last updated about 2 hours ago.

↻ UPDATE

Syncing the Schema:

Click "Get latest version" to fetch the schema via introspection query.

Then, click "Update" (top right) to apply the new schema.

Upstream Authentication (if protected):

Navigate to: API > Advanced Options > Upstream Auth Headers

Add your Authorization Header (e.g., bearer token or basic auth).

Field Based Permissions

Understanding the GraphQL Schema

```
type Query {  
  accounts: [Account!]  
}
```

```
type Account {  
  owner: String!  
  number: ID!  
  balance: Float!  
}
```

- Query: The entry point of your API
 - accounts returns a list of Account objects
- Account Type:
 - owner: Name of account holder
 - number: Unique account number
 - balance: Sensitive field showing current balance

Field Based Permissions

Why Use Field-Based Permissions?

Not every API consumer should see everything.

Internal user can query:

```
query {  
  accounts {  
    owner  
    number  
    balance  
  }  
}
```

External user tries to access `balance` and gets:

```
{  
  "errors": [  
    {  
      "message": "field: balance is restricted on  
type: Account"  
    }  
  ]  
}
```

Setting Up Field Based Permissions

Restricted and allowed types and fields can also be set up via Tyk Dashboard.

Optional: Configure a Policy from System Management > Policies > Add Policy.

- From System Management > Keys > Add Key select a policy or configure directly for the key.
- Select your GraphQL API (marked as GraphQL).
- Enable either Block list or Allow list. By default, both are disabled. It's not possible to have both enabled at the same time - enabling one switch automatically disables the other.

Setting Up Field Based Permissions - Blocklist

By default all Types and Fields will be unchecked. By checking a Type or Field you will disallow to use it for any GraphQL operation associated with the key.

For example, the settings illustrated below would block the following:

- code and countries fields in Continent type.
- latt and longt fields in Coordinates type.

Types

Filter Types

Types list SELECT ALL DESELECT ALL

- ☐ Country
- ☒ Coordinates
- ☒ Continent
- ☐ Language
- ☐ State
- ☐ Query

Fields

Filter field names

Coordinates SELECT ALL DESELECT ALL

- ☒ latt
- ☒ longt

Continent SELECT ALL DESELECT ALL

- ☒ code
- ☐ name
- ☒ countries

Setting Up Field Based Permissions - Allowlist

By default all Types and Fields will be unchecked. By checking a Type or Field you will allow it to be used for any GraphQL operation associated with the key.

For example, the settings illustrated below would only allow the following:

- code field in `Continent` type.
- code and name fields in `Language` type.

Types list [SELECT ALL](#) | [DESELECT ALL](#)

- ☐ Country
- ☐ Coordinates
- ☒ Continent
- ☒ Language
- ☐ State
- ☐ Query



☐ longt

Continent [SELECT ALL](#) | [DESELECT ALL](#)

- ☒ code
- ☐ name
- ☐ countries

Language [SELECT ALL](#) | [DESELECT ALL](#)

- ☒ code
- ☒ name



Hands-On

Setting Up Field Based Permissions

Setting Up Field Based Permissions

1. Create a policy
2. Create an API Key with policy
3. Set up blocklist on the `Country Type` field
4. Use the Playground and add the header as a json:

```
{  
  
  "Authorization": "API-KEY"  
  
}
```

1. Make the query:

```
query {  
  
  countries {  
  
    code  
  
    name  
  
    emoji  
  
  }  
  
}
```

Complexity Limiting

GraphQL Complexity Limiting

Even with a simple schema, deeply nested or resource-heavy operations can overload your upstream services.

```
type Query {  
  continents: [Continent!]!  
}
```

```
type Continent {  
  name: String!  
  countries: [Country!]!  
}
```

```
type Country {  
  name: String!  
  continent: Continent!  
}
```

Complexity Limiting

Deeply Nested Query Example

```
{  
  continents {  
    countries {  
      continent {  
        countries {  
          continent {  
            countries {  
              name  
            }  
          }  
        }  
      }  
    }  
  }  
}
```

This query is valid, but heavily recursive and can stress your backend.

Complexity

Limiting

Limit query depth by setting a max depth in:

- A Policy
- An Individual Key

Query Depth = 2

```
{  
  
  continents {  
  
    name  
  
  }  
  
}
```

Query Depth = 3

```
{  
  
  continents {  
  
    countries {  
  
      name  
  
    }  
  
  }  
  
}
```

When Depth Limit is Exceeded:

Tyk will block the query and respond with:

```
{  
  
  "error": "depth limit  
exceeded"  
  
}
```

Introspection

- GraphQL introspection allows clients to query a server for its schema.
- Useful for tools like GraphQL Playground and Tyk Dashboard to understand available types and operations.
- Tyk uses introspection to fetch the upstream schema and render it in the UI when setting up a GraphQL proxy.

 Important: Introspection reflects the upstream schema only, not Tyk's schema transformations.

Introspection

Details

Upstream URL

https://countries.trevorblades.com/

☒ Upstream Protected

Upstream protected by

☒ Headers

☒ Certificate

☐ Persist headers for future use

Introspection headers

Key

Header name

Value

Header value

ADD

Domain

Add Certificate

SELECT CERTIFICATE

ENTER CERTIFICATE ID

Certificate

Select certificate

Introspection

GraphQL Introspection in Production

- Introspection is useful for development and debugging.
- In production, introspection can:
 - Reveal sensitive schema and implementation details.
 - Allow attackers to discover and map out your API.
- If Authentication Mode is Open (Keyless):
 - Introspection is always enabled and cannot be disabled.
- You can disable introspection per key or via security policy using:
 - Tyk Dashboard
 - Tyk Gateway API

Introspection Queries

- Any GraphQL API can be explored using introspection queries.
- Introspection queries reveal detailed schema information.
- Example: Introspecting all types in the schema.

```
query {  
  __schema {  
    types {  
      name  
      description  
      kind  
    }  
    queryType {  
      fields {  
        name  
        description  
      }  
    }  
  }  
}
```

Introspection Queries

- **Schema Overview Query**
 - Retrieves a summary of all types, queries, and mutations in the schema, including their names, kinds, and descriptions.
- **Single Type Details Query**
 - Fetches detailed information about a specific type, such as its fields, arguments, interfaces, enum values, and descriptions.
- **Field Arguments Query**
 - Focuses on the arguments of a specific field, detailing their names, types, and default values.
- **Enum Values Query**
 - Retrieves the possible values for an enum type, including descriptions and deprecation status.
- **Interface and Union Types Query**
 - Provides information on the possible types that implement an interface or belong to a union.

Validation - Query Validation

- Tyk supports validation of GraphQL queries and schemas to prevent errors during request processing.
- Query Validation:
 - Tyk's native GraphQL engine validates queries against the GraphQL Specification.
 - Both the Gateway in front of your existing GraphQL API and any Universal Data Graph have validation middleware.
- Invalid requests are caught by the Gateway and not forwarded upstream, returning errors to the client.

Validation - Schema Validation

- Schema Validation:
- Prevents broken or invalid schemas from being saved or updated in Tyk.
- Available only via Tyk Dashboard or Dashboard API.
- Ensures schemas do not contain:
 - Duplicated operation types (Query, Mutation, Subscription)
 - Duplicated type names, field names, or enum values
- Usage of unknown types
- If schema validation fails via the API, a 400 Bad Request is returned with detailed errors.

Introspection Headers

- Purpose: Used when Tyk Dashboard sends an introspection query to fetch the upstream GraphQL schema.
- Usage scenario: Required when upstream GraphQL APIs are protected and require authorization for schema discovery.
- Key Points:
 - Headers are only used during schema introspection from the Dashboard.
 - These headers are not forwarded during runtime API requests from consumers.
 - Configurable in the Advanced Options tab or via raw API definition:

Introspection Headers

```
"graphql": {  
  "execution_mode": "proxyOnly",  
  "proxy": {  
    "auth_headers": {  
      "admin-auth": "token-value"  
    }  
  }  
}
```

Request Headers

- Purpose: Used to inject additional headers into runtime requests proxied through Tyk Gateway.
- Usage scenario: Add metadata, tokens, or dynamic values to upstream requests.
- Key Points:
 - Only active during actual API requests, not during introspection.
 - Supports static values or dynamic context variables like `$tyk_context.*`.
 - Configurable in the Advanced Options tab or raw API definition:

Request Headers

```
"graphql": {  
  "execution_mode": "proxyOnly",  
  "proxy": {  
    "request_headers": {  
      "context-vars-metadata": "$tyk_context.path",  
      "static-metadata": "static-value"  
    }  
  }  
}
```

Persisting GraphQL queries

- Tyk allows exposing a GraphQL query as a REST endpoint.
- Achieved using `persist_graphql` inside `extended_paths`.
- Configured on an HTTP-type API definition.
- Useful for:
 - Creating simple, cacheable REST endpoints
 - Predefined access to known GraphQL operations

Requirements:

- The `target_url` must point to a GraphQL upstream.
- `enable_context_vars` must be set to `true`.

Persisting GraphQL queries

API Definition & Versioning Setup

- Basic API definition:

```
"proxy": {  
  "listen_path": "/trevorblades/",  
  "target_url": "https://countries.trevorblades.com",  
  "strip_listen_path": true  
}
```

- Enable versioning with:
 - `use_extended_paths: true`
 - `not_versioned: true`
- GraphQL to REST middleware lives inside `extended_paths.persist_graphql`.

Persisting GraphQL queries

The vital part of this is the `extended_paths.persist_graphql` **field**

Sample `persist_graphql` configuration:

```
"persist_graphql": [  
  {  
    "method": "GET",  
    "path": "/getContinentByCode",  
    "operation": "query ($continentCode: ID!) { continent(code: $continentCode)  
{ code name countries { name } } }",  
    "variables": {  
      "continentCode": "EU"  
    }  
  }  
]
```

Persisting GraphQL queries

Each entry includes:

- method – e.g., GET
- path – e.g., /getContinentByCode
- operation – the GraphQL query
- variables – parameters passed to the query

```
{
  "data": {
    "continent": {
      "code": "EU",
      "name": "Europe",
      "countries": [
        {
          "name": "Andorra"
        },
        ...
      ]
    }
  }
}
```

Dynamic Variables

You can pass dynamic variables to your persisted GraphQL queries.

- Supported sources:
 - Request headers
 - URL path parameters
- Use Tyk context variables to reference them.
- Ideal for flexible query inputs without hardcoding values.

Dynamic Variables

Example: Extract countryCode from request header code

Request Header:

code: UK

```
{
  "method": "GET",
  "path": "/getCountryByCode",
  "operation": "query ($countryCode: ID!) {\n
country(code: $countryCode) {\n          code\n
name\n    }\n}",
  "variables": {
    "countryCode": "$tyk_context.headers_code"
  }
}
```

Dynamic Variables

Response:

```
{  
  "data": {  
    "country": {  
      "code": "UK",  
      "name": "United Kingdom"  
    }  
  }  
}
```

Dynamic Variables

Example: Extract countryCode from the URL

```
{  
  "method": "GET",  
  "path": "/getCountryByCode/{countryCode}",  
  "operation": "query ($countryCode: ID!) {\n  
country(code: $countryCode) {\n          code\n  
name\n    }\n}",  
  "variables": {  
    "countryCode": "$path.countryCode"  
  }  
}
```

Request URL:

/getCountryByCode/NG

Dynamic Variables

Response:

```
{  
  "data": {  
    "country": {  
      "code": "NG",  
      "name": "Nigeria"  
    }  
  }  
}
```

Module 3

Universal Data Graph

What is the Universal Data Graph (UDG)?

Universal Data Graph (UDG) is Tyk's powerful GraphQL execution engine that allows you to create a unified GraphQL API by stitching together multiple data sources into a single, cohesive schema.

Core Concept:

- Define a single GraphQL schema that represents your ideal API

- Map schema fields to different backend data sources

- Tyk's execution engine automatically resolves queries across all data sources

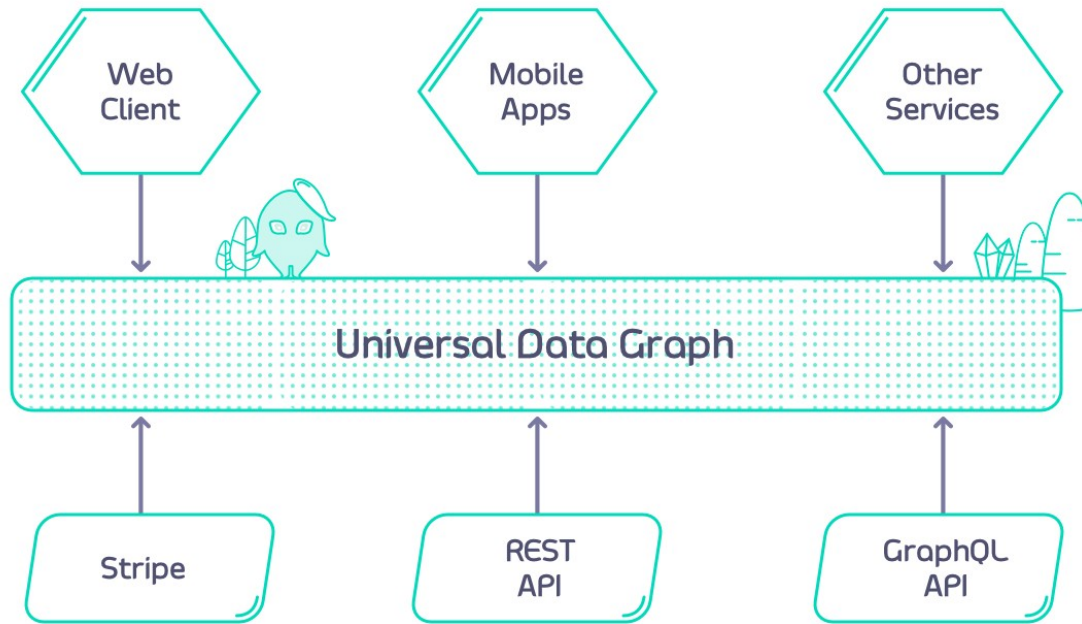
- No need to modify existing backend services

Execution Mode:

- UDG uses the `executionEngine` execution mode

- Operates as a GraphQL gateway that orchestrates data fetching

- Supports multiple data source types simultaneously



Currently supported DataSources:

- REST
- GraphQL
- SOAP (through the REST datasource)
- Kafka

Key Concepts - DataSources

- Resolvers vs. DataSources
 - Resolvers: Functions that return data for a field (typical in GraphQL)
 - DataSources: Config-driven way to fetch data for fields
 - No code required — just configuration
-  Types of DataSources in Tyk
- Internal:

REST/SOAP APIs already managed in Tyk

→ Use middleware to validate and transform data
- External:

APIs not (yet) managed in Tyk

→ Easily included in your data graph

→ Can transition to internal when needed

Key Concepts - Arguments

- GraphQL Arguments → REST Calls

Schema example:

```
type Query {  
    user(id: Int!): User  
}
```

```
type User {  
    id: Int!  
    name: String  
}
```

Goal: Map the id argument in a GraphQL query to the correct REST API path

Key Concepts - Arguments

Injecting Arguments into REST Paths

- Templating REST URLs
- In the “Configure Data Source” tab:

Use this template in the path field:

```
https://example.com/user/{{ .arguments.id }}
```

Type { to access a dropdown of available fields and arguments.

Tip: You can dynamically inject any GraphQL argument or field this way.

Key Concepts - Arguments

Configure data source

QUERY > USER

Data source details

Data source name

My Data Source

URL

https://www.my-data-source/example/

Search parameter

GraphQL schema arguments will appear in blue

id : Argument id

user : user of type Query

Field mapping

☒ Disable field mapping

If you type an opening curly brace ({), the parameter dropdown will appear

GraphQL schema arguments will appear in blue

GraphQL schema object fields will appear in orange

Key Concepts - Field Mappings

Field Mappings – When Are They Needed

Automatic Field Mapping

- When the GraphQL schema mirrors the REST API response, no manual field mapping is required.

Example REST response:

```
{  
  "id": 1,  
  "name": "Martin Buhr"  
}
```

Matching GraphQL schema:

```
type Query {  
  user(id: Int!): User  
}  
  
type User {  
  id: Int!  
  name: String  
}
```

Key Concepts - Field Mappings

Field Mappings – Handling Different Field Names

- If the REST response uses a different field name, field mapping is required.

Example REST response:

```
{
  "id": 1,
  "user_name": "Martin Buhr"
}
```

GraphQL schema:

```
{
  "id": 1,
  "user_name": "Martin Buhr"
}
```

In this case, manual mapping is needed:

- Uncheck "Disable field mapping"
- Set `name` field path to `user_name`
- Nested fields can use dot notation: `user.full_name`

Key Concepts - Reusing response fields

Chaining Data Across APIs

You can use data from one API response to call another API.

Example APIs:

- People List: <https://people-api.dev/people>
- Person Details: https://people-api.dev/people/{person_id}
- Driver Licenses: https://driver-license-api.dev/driver-licenses/{driver_license_id}

Key Concepts - Defining Data Source URLs

Static and Dynamic URL Templates

- **Query.people**

Static URL for retrieving the list of people:

```
https://people-api.dev/people
```

- **Query.person**

Uses the id argument dynamically in the URL:

```
https://people-api.dev/people/{{.arguments.id}}
```

- **Person.driverLicense**

Uses data from the parent object via .object placeholder:

```
https://driver-license-api.dev/driver-licenses/{{.object.driverLicenseID}}
```

Reminder:

- Use `.object` to reference fields from the parent object.
- Use `.arguments` to reference query arguments.

Key Concepts - Defining Data Source URLs

GraphQL Query:

```
{
  people {
    id
    name
    age
    driverLicense {
      id
      issuedBy
      validUntil
    }
  }
}
```

Response Example:

```
{
  "data": {
    "people": [
      {
        "id": 1,
        "name": "User 1",
        "age": 40,
        "driverLicense": {
          "id": "DL1234",
          "issuedBy": "United Kingdom",
          "validUntil": "2040-01-01"
        }
      },
      {
        "id": 2,
        "name": "User 2",
        "age": 30,
        "driverLicense": {
          "id": "DL5555",
          "issuedBy": "United Kingdom",
          "validUntil": "2035-01-01"
        }
      }
    ]
  }
}
```

UDG Header Management

Two levels of header control:

Global Headers

- Defined in `graphql.engine.global_headers`
- Sent to all data sources
- Can include request context variables

Data Source Headers

- Defined in `graphql.engine.data_sources[].config.headers`
- Specific to individual data sources
- Also supports context variables like JWT claims

```
{
  "global_headers": [
    { "key": "global-header", "value": "example-value" },
    { "key": "request-id", "value": "$tyk_context.request_id" }
  ]
}
```

UDG Header Management

Data Source Headers and Context Variables

Example - Data Source Header Config:

```
{  
  "headers": {  
    "data-source-header": "data-source-header-value",  
    "datasource1-jwt-claim": "$tyk_context.jwt_claims_datasource1"  
  }  
}
```

Key Notes:

- Defined per data source
- Ideal for individual API authentication needs
- Can also access JWT claims and request context values

UDG Header Management

Header Precedence Rules

When header keys overlap, Tyk applies priority:

Header name	Header value	Defined on level
example-header	data-source-value	data source
datasource1	\$tyk_context.jwt_claims_datasource1	data source
request-id	\$tyk_context.request_id	global

Data Source header overrides Global header if both define the same key.

Module 4

DataSources

Connect Data Sources

Unified Data Graph (UDG) – Datasource Overview

- **Datasources power your UDG and its schema**
 - **Can be attached to any field in the UDG schema**
 - **Support nested configuration**
- **Quick Start vs Full Control**
 - **You can add datasources directly to UDG without registering them as APIs in Tyk**
 - **For advanced features (rate limits, quotas, transformations), import the API into Tyk first**
- **Supported Datasource Types:**
 - **REST**
 - **GraphQL**
 - **SOAP (via REST Datasource)**
 - **Kafka**

GraphQL Datasource Overview

- Makes GraphQL queries to upstream GraphQL services
- Configured similarly to REST Datasource
- Handles query and response specifics automatically

Schema Example:

```
type Query {  
  employee(id: Int!):  
    Employee  
}  
type Employee {  
  id: Int!  
  name: String!  
}
```

Example Query:

```
query TykCEO {  
  employee(id: 1) {  
    id  
    name  
  }  
}
```

GraphQL Datasource Overview

- **GraphQL Response Parsing**

Actual Response:

```
{
  "data": {
    "employee": {
      "id": 1,
      "name": "Martin
Buhr"
    }
  }
}
```

Tyk Extracts:

```
{
  "employee": {
    "id": 1,
    "name": "Martin Buhr"
  }
}
```

GraphQL Datasource Overview

REST vs GraphQL

- REST APIs do not wrap responses
- GraphQL APIs wrap responses in root fields (like data, employee)
- GraphQL field mapping is enabled by default

Why Use GraphQL Datasource?

- Specification-compliant GraphQL requests
- Supports nested REST & GraphQL APIs in a single schema
- Query planner sends correct queries to upstreams

How the Query Planner Works

- Checks if a field has a datasource
- That datasource resolves the field and its children
- If a nested field has its own datasource, ownership shifts

GraphQL Datasource Overview

Summary

- Tyk supports full GraphQL Datasource configuration
- Automatic unwrapping of data field
- Query planner handles mixed datasources

Kafka Datasource Overview

- Enables subscribing to Kafka topics and querying events with GraphQL.
- Integrates Kafka's powerful pub/sub model into your Universal Data Graph.
- Operates based on Kafka's consumer group mechanism.

Kafka Consumer Groups

- A group of cooperating consumers that share processing load.
- Kafka automatically manages membership and rebalances partitions.
- Allows for horizontal scaling and fault tolerance.

Kafka Datasource Overview

Basic Kafka Configuration

- `broker_addresses`: Kafka broker hosts
- `topics`: Kafka topics to subscribe to
- `group_id`: Consumer group identifier
- `client_id`: Optional identifier for logging and debugging

Kafka Datasource Overview

Configuring via Tyk Dashboard

- Click the field to attach Kafka DataSource.
- In Configure Data Source, choose KAFKA.
- Provide:
 - Data source name
 - Broker addresses
 - Topics
 - Group ID
 - Client ID
 - (Optional) Kafka version, balance strategy, field mappings
- Click Save.

Kafka Datasource Overview

Configure data source

[PRODUCTINFO > DESCRIPTION](#)

Select data source type

Choose from existing Tyk APIs :

▼ GraphQL | Tyk

69

▼ REST | Tyk

18

Or, choose from data sources previously configured in this UDG :

▼ Data Sources | Tyk

Or, add a new external data source

REST

GRAPHQL

KAFKA



Kafka Datasource Overview

Once done the field you just configured will show information about data source type and name:



The screenshot displays a configuration interface with two sections: 'Type | Query' and 'Type | Subscription'. The 'Type | Query' section contains a single entry 'topProducts' with a teal dot, followed by the text 'topProducts | GraphQL' in teal. The 'Type | Subscription' section contains a single entry 'stock' with a teal dot, followed by the text 'kafka-consumer-group | Kafka' in teal. A teal arrow points from the 'stock' entry in the 'Type | Subscription' section to the 'kafka-consumer-group' text.

Type Query
● topProducts GraphQL

Type Subscription
● stock kafka-consumer-group Kafka

Kafka Datasource Overview

GraphQL Subscriptions with Kafka

Real-Time Subscriptions

Use GraphQL subscription queries to listen to Kafka events.

Define Subscription type in your schema:

```
type Product {  
  name: String  
  price: Int  
  inStock: Int  
}  
  
type Subscription {  
  productUpdated: Product  
}
```

Subscribers will get real-time updates on every event.

Example Subscription Query

```
subscription Products {  
  productUpdated {  
    name  
    price  
    inStock  
  }  
}
```

- Automatically receives new product data when Kafka publishes an event.
- Compatible with most GraphQL clients.

Kafka Datasource Overview

Publishing Test Events

Simulating Kafka Events

Publish event to product-updates topic:

```
{
  "productUpdated": {
    "name": "product1",
    "price": 1624,
    "inStock": 219
  }
}
```

- Use any Kafka producer or GUI tool.
- Useful for testing subscriptions and schema mappings.

Kafka Datasource Overview

Summary

Kafka DataSource in Tyk

- Seamlessly integrates Kafka with GraphQL APIs.
- Allows real-time streaming via GraphQL subscriptions.
- Scalable via Kafka consumer groups.
- Easy setup via Tyk Dashboard or API Definition.

REST Datasource Overview

- REST DataSources connect REST APIs to your GraphQL schema
- Enables field-level resolution via REST endpoints
- Supported for both external APIs and Tyk-managed APIs

REST Datasource Overview

- Select the field in your schema
- Configure as External REST DataSource
 - Name
 - URL
 - Method (GET, POST, etc.)
 - Optional: Headers, Field Mapping

Configure data source

QUERY > GETACTIVEUSERS

Data source details

Data source name

getActiveUsers_Query

URL

Enter data source url

Method

Select method type for your data source

☒ Add headers

ADD

Headers

HEADER KEY	HEADER VALUE	DELETE
Enter header key	Enter header value	

Field mapping

☐ Use default field mapping

Path

Enter path

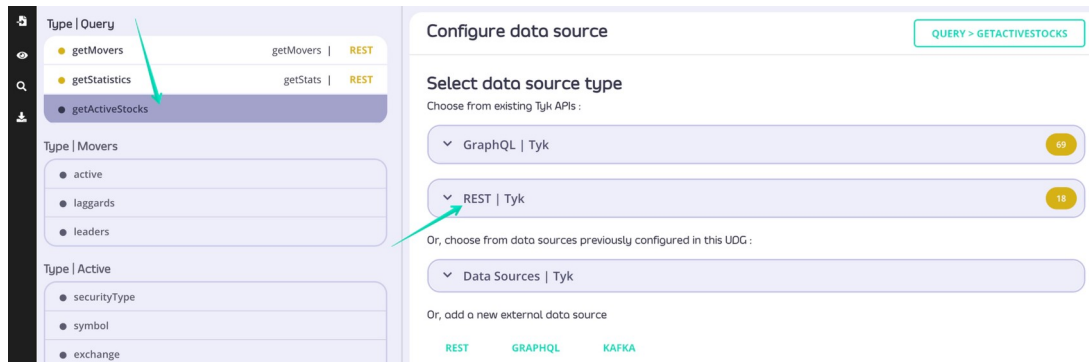
RESET

SAVE AND UPDATE API

REST Datasource Overview

Attaching Tyk REST API

- Select the field
- Choose REST | Tyk from the dropdown
- Pick from existing Tyk APIs
- Provide endpoint and method
- Optional headers and mapping



Save and Generate Resolver

- Click "Save & Update API"
- Resolver is auto-generated
- GraphQL runtime will call the REST endpoint when the field is queried

REST DataSource Overview

Auto-generate REST DataSource via OAS

- Use Tyk Dashboard API
- Endpoint: `POST /api/data-graphs/data-sources/import`
- Request Body:

```
{  
  "type": "string",  
  "data": "string"  
}
```

`type` is an enum with the following possible values:

- `openapi`
- `Asyncapi`

To import an OAS specification you need to choose `openapi`

REST Datasource Overview

Summary

- REST DataSources make it easy to integrate legacy REST APIs
- Both external and internal APIs are supported
- Automatic configuration possible with OAS



Demo

Stitching 2 Data Sources into a single UDG

UDG Countries Enriched — Overview

The "UDG Countries Enriched" API is a Universal Data Graph (UDG) example that combines data from two sources into a single GraphQL endpoint.

It exposes a unified GraphQL API that merges:

- GraphQL Countries API (internal Tyk API) — provides code and name
- REST Countries API (external service) — provides population

UDG Countries Enriched — Overview

GraphQL schema

```
type Country {  
  code: ID!  
  
  name: String!  
  
  pop: Int!    # This comes from the REST API  
}  
  
type Query {  
  country(code: ID!): Country  
}
```

How it works

When you make a query like:

```
query {  
  country(code: "GB") {  
    code  
    name  
    pop  
  }  
}
```

UDG Countries Enriched — How it works

The UDG engine:

1. Calls the GraphQL Countries API (internal):
 - a. Uses `tyk://8ba9f26729e041a95f84f01dd0ce6d9b/`
 - b. Passes code: "GB" as an argument
 - c. Gets back: `{ code: "GB", name: "United Kingdom" }`
2. Calls the REST Countries API (external):
 - a. URL: `http://countries-rest-api:4100/cca2/{ {.object.code} }`
 - b. Uses `{ {.object.code} }` from the GraphQL response (dynamic templating)
 - c. Gets back: `{ population: 67081000 }`
3. Merges the results:
 - a. Maps `population` → `pop` (field mapping)
 - b. Returns: `{ code: "GB", name: "United Kingdom", pop: 67081000 }`

Data sources configuration

Data Source 1: GraphQL Countries (Internal)

- Kind: GraphQL
- Internal: true (uses `tyk://` protocol to call another Tyk API)
- Provides: `code`, `name`
- Root fields: `Query.country`

Data Source 2: REST Countries (External)

- Kind: REST
- Internal: false
- URL: `http://countries-rest-api:4100/cca2/{{.object.code}}`
- Provides: `population` (mapped to `pop`)
- Root fields: `Country.pop`

Data sources configuration

Features

- Dynamic templating: `{{.object.code}}` uses data from the first data source
- Field mapping: `population` → `pop`
- Internal API calls: uses `tyk://` to call another Tyk API
- External API calls: can call any REST endpoint
- Single request: client makes one query, UDG handles multiple upstream calls

Example query

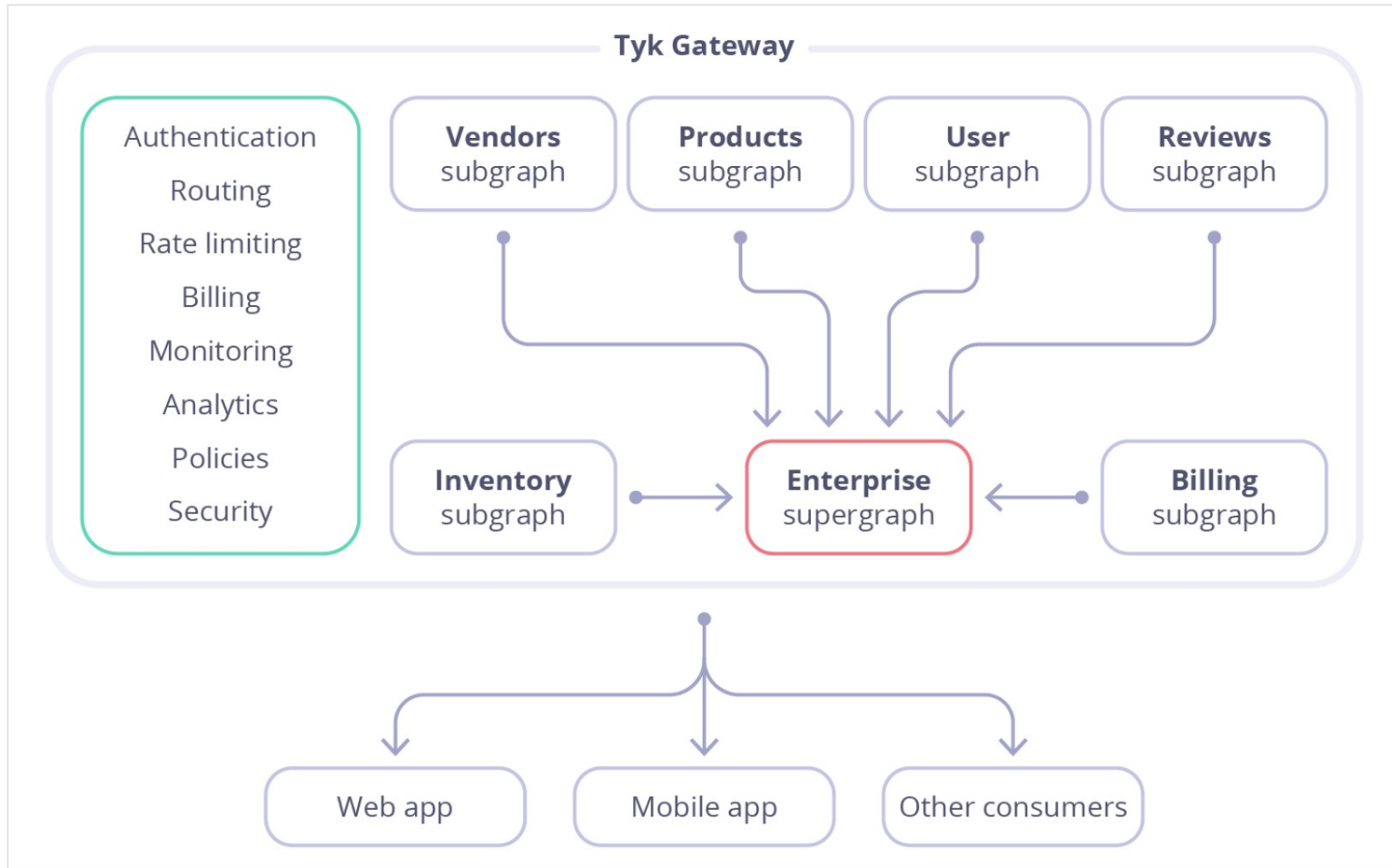
```
curl -X POST http://tyk-gateway.localhost:8080/universal-data-graph-composed/ \
-H "Content-Type: application/json" \
-d '{
  "query": "query { country(code: \"GB\") { code name pop } }"
}'
```

Module 2

GraphQL Federation

What is GraphQL Federation

- GraphQL Federation solves the complexity of scaling GraphQL in large enterprises.
- Allows you to break down monolithic GraphQL services into multiple backend services.
- Tyk (v4.0+) supports federation to expose multiple services as a single, unified graph.
- Simplifies maintenance and encourages service ownership.



Subgraphs and Supergraphs

- Subgraph is a representation of a back-end service and defines a distinct GraphQL schema. It can be queried directly as a separate service or it can be federated into a larger schema of a supergraph.
- Supergraph is a composition of several subgraphs that allows the execution of a query across multiple services in the backend.

Subgraph Examples

Users

```
extend type Query {  
  me: User  
}  
  
type User  
@key(fields: "id") {  
  id: ID!  
  username: String!  
}
```

Products

```
extend type Query {  
  topProducts(first: Int = 5): [Product]  
}  
  
extend type Subscription {  
  updatedPrice: Product!  
  updateProductPrice(upc: String!): Product!  
  stock: [Product!]  
}  
  
type Product @key(fields: "upc") {  
  upc: String!  
  name: String!  
  price: Int!  
  inStock: Int!  
}
```

Subgraph Examples

Reviews

```
type Review {  
  body: String!  
  author: User! @provides(fields: "username")  
  product: Product!  
}
```

```
extend type User @key(fields: "id") {  
  id: ID! @external  
  username: String! @external  
  reviews: [Review]  
}
```

```
extend type Product @key(fields: "upc") {  
  upc: String! @external  
  reviews: [Review]  
}
```

After federating all subgraphs in Tyk, the unified supergraph schema looks like:

```
type Query {  
  topProducts(first: Int = 5): [Product]  
  me: User  
}  
  
type Subscription {  
  updatedPrice: Product!  
  updateProductPrice(upc: String!): Product!  
  stock: [Product!]  
}  
  
type Review {  
  body: String!  
  author: User!  
  product: Product!  
}  
  
type Product {  
  upc: String!  
  name: String!  
  price: Int!  
  inStock: Int!  
  reviews: [Review]  
}  
  
type User {  
  id: ID!  
  username: String!  
  reviews: [Review]  
}
```

Subgraph and Supergraphs

- Split your system across multiple subgraphs for modularity
- Use `@key`, `@external`, and `@provides` to link types across services
- All subgraphs unified and governed via Tyk Gateway
- Benefits: Faster development, clear service ownership, centralized security
- One supergraph = One entry point for all consumers

Creating a Subgraph via the Dashboard UI

- Log in to Dashboard → APIs > Add New API > Federation > Subgraph
- Choose a name and provide an upstream URL
- If upstream is protected, select "Upstream Protected" and provide auth details (Header or Certificate)

Overview

Name

Type

☐  Rest

☐  TCP

☐  GraphQL

☐  UDG

☒  Federation

Federation

☒  Subgraph

☐  Supergraph
0 Subgraphs available

Details

Upstream URL

☐ Upstream Protected

Details

Upstream URL

☒ Upstream Protected

Upstream protected by

☒ Headers

☐ Certificate

☐ Persist headers for future use

Header name

Header value

Creating a Supergraph via the Dashboard UI

- Go to APIs > Add New API > Federation > Supergraph
 - In Details, select subgraphs to include
 - Configure supergraph settings like any other API
 - Save → supergraph is available in the API list

Overview

Name

Type

☐  Rest

☐  TCP

☐  GraphQL

☐  UDG

☒  Federation

Federation

☐  Subgraph

☒  Supergraph
3 Subgraphs available

Details

Subgraphs

Details

Subgraphs

☒ subgraph-reviews

☒ subgraph-products

☒ subgraph-users

▼

All sub graphs

☒ subgraph-reviews

☒ subgraph-products

☒ subgraph-users

Defining Global Headers for Supergraphs

- Open supergraph API in Dashboard
 - Go to Subgraphs tab → Global Headers
 - Add header name and value
 - Click Update, then Update API to save changes
 - Headers are forwarded to all relevant subgraphs

super_fed

VIEW RAW DEFINITION

OPTIONS ▾

UPDATE

Core Settings Subgraphs Schema Playground Advanced Options

▾ Global Headers

Subgraphs

ADD SUBGRAPH

▾ products_fed



▾ accounts_fed



▾ reviews_fed



Defining Global Headers for Supergraphs

- Open supergraph API in Dashboard
 - Go to Subgraphs tab → Global Headers
 - Add header name and value
 - Click Update, then Update API to save changes
 - Headers are forwarded to all relevant subgraphs

super_fed

VIEW RAW DEFINITION

OPTIONS ▾

UPDATE

Core Settings Subgraphs Schema Playground Advanced Options

▾ Global Headers

Subgraphs

ADD SUBGRAPH

▾ products_fed



▾ accounts_fed



▾ reviews_fed



Entities – Defining the Base Entity

- Base entity must have @key directive with fields that uniquely identify it
- Multiple primary keys allowed
- Example:

```
type MyEntity @key(fields: "id") @key(fields: "name") {  
  id: ID!  
  name: String!  
}
```

Entities – Extending Entities

- Extensions add fields to base entities in other subgraphs
- Must use @key with the same primary key fields
- Primary keys referenced with @external directive
- Example:

```
extend type MyEntity @key(fields: "id") {  
  id: ID! @external  
  newField: String!  
}
```

Entities – Entity Stubs

- Stubs reference an entity without adding fields
- Minimal info needed to identify the entity
- Example:

```
extend type MyEntity @key(fields: "id") {  
  id: ID! @external  
}
```

Shared Types & Extending Shared Types

- Shared types have the same name and structure in multiple subgraphs
- Extensions allowed only if normalized resolutions are identical
- Example enums with extensions:

Subgraph 1

```
enum Example { A, B }  
extend enum Example { C }
```

Subgraph 2:

```
enum Example { A, B, C }
```

Extension Orphans

- Orphans occur when extensions are unresolved in federation
- Happens if base type is missing or defined in multiple subgraphs
- Example error: extending a type defined in zero or multiple subgraphs

```
extend type Person {  
  name: String!  
}
```

- Federation fails with an error